

AGRIVISTA FIELD

Dossier de conception

Mise en situation professionnelle - Développement full stack mobile Flutter

Formation	Master BIHAR
Technologie	Flutter - Dart 3
Projet	Application mobile de gestion d'interventions terrain
Auteur	ARGAILLOT Dorian
Session	2025-2026

Finalité du document

Présenter et justifier la conception de la première version d'AgriVista Field avant son développement : périmètre, usages, écrans, modèle de données, architecture, choix techniques et stratégie de persistance locale.

1. Reformulation du besoin et périmètre retenu

1.1 Contexte professionnel

AgriVista installe et maintient des stations de capteurs connectés pour des exploitations agricoles et viticoles corses. Les techniciens interviennent sur le terrain, parfois dans des zones où la couverture réseau est faible ou intermittente. La première version de l'application interne « AgriVista Field » doit remplacer une gestion papier des interventions et fournir un outil mobile simple, fiable et exploitable sur Android et iOS.

1.2 Besoin métier reformulé

Le technicien doit pouvoir consulter les interventions fournies par la source JSON de référence, les rechercher et les filtrer, ouvrir le détail d'une intervention et faire évoluer son statut au cours de l'intervention. Les changements de statut sont enregistrés localement afin de survivre à la fermeture ou au redémarrage de l'application. La synchronisation avec un serveur central n'appartient pas à cette version.

1.3 Objectifs fonctionnels

- Charger et désérialiser la liste des interventions depuis le JSON fourni.
- Afficher une liste claire avec recherche et filtres par statut et priorité.
- Afficher le détail : station, domaine/localisation, description, statut et historique.
- Autoriser uniquement la progression planifiée → en_cours → terminée.
- Conserver localement les statuts modifiés après redémarrage.
- Présenter explicitement les états chargement, succès et erreur.
- Fournir un thème Material 3 cohérent et un écran de profil simple.

1.4 Périmètre retenu

Élément	Décision	Précision
Chargement JSON	Inclus	Lecture distante en lecture seule et désérialisation typée.
Liste	Inclus	Recherche + filtres statut/priorité.
Détail	Inclus	Toutes les informations demandées + historique.
Statut	Inclus	Transition vers l'état suivant uniquement.
Persistance	Inclus	Surcharge locale du statut par identifiant.
États UI	Inclus	loading / data / error + action Réessayer.
Thème & profil	Inclus	Material 3 + profil du technicien.
Backend / synchro	Exclu	Prévu pour une version ultérieure.
Authentification distante	Exclu	Aucun backend demandé.
Fonctions bonus	Après socle	Sombre/clair, tablette, tri, photo, note, carte, dashboard, animations.

Hypothèse de conception

La recherche portera sur la station, le domaine et la description. Le JSON reste la source de référence et ne sera jamais modifié. En cas de changement de statut, seule une surcharge locale est enregistrée.

2. Acteurs, cas d'usage et user stories

2.1 Acteur principal

Le seul acteur humain nécessaire au périmètre minimum est le technicien de terrain. Le fichier JSON distant joue le rôle de source de données externe en lecture seule ; le stockage local conserve les modifications de statut.

2.2 User stories principales

ID	Acteur	En tant que... je veux...	Afin de...
US01	Technicien	Je veux charger mes interventions	connaître les opérations à réaliser.
US02	Technicien	Je veux rechercher une intervention	la retrouver rapidement sur le terrain.
US03	Technicien	Je veux filtrer par statut et priorité	me concentrer sur les interventions pertinentes.
US04	Technicien	Je veux consulter le détail	connaître station, localisation, description et historique.
US05	Technicien	Je veux démarrer une intervention planifiée	indiquer qu'elle est en cours.
US06	Technicien	Je veux terminer une intervention en cours	mettre à jour son avancement.
US07	Technicien	Je veux retrouver mes statuts après redémarrage	ne pas perdre mon travail terrain.
US08	Technicien	Je veux connaître clairement un problème de chargement	pouvoir réessayer sans écran bloqué.
US09	Technicien	Je veux consulter mon profil	identifier le technicien associé aux données.

2.3 Cas d'usage

ID	Cas d'usage	Déclencheur	Résultat attendu
UC01	Charger les interventions	Démarrage / rechargement	Liste fusionnée avec les statuts locaux.
UC02	Consulter / filtrer la liste	Données disponibles	Sous-ensemble correspondant aux critères.
UC03	Consulter une intervention	Sélection d'une carte	Écran de détail complet.
UC04	Mettre à jour le statut	Statut planifié ou en cours	Nouveau statut enregistré localement.
UC05	Restaurer les statuts	Redémarrage de l'app	Les modifications locales sont réappliquées.
UC06	Consulter le profil	Ouverture de l'onglet profil	Identité du technicien affichée.

3. Scénarios principaux et parcours utilisateur

3.1 Scénario nominal - démarrage et consultation

1. L'application démarre et affiche un état de chargement.
2. La couche Data récupère le JSON distant via Dio.
3. Les données sont désérialisées en objets typés.
4. Les éventuels statuts locaux sont lus dans Hive puis appliqués aux interventions correspondantes.
5. Riverpod expose la liste finale à la Presentation.
6. Le technicien recherche ou filtre une intervention, puis ouvre son détail.

3.2 Scénario - changement de statut

1. Depuis le détail, le technicien appuie sur l'action correspondant à la transition autorisée.
2. Le Use Case vérifie la transition : planifiée → en_cours ou en_cours → terminée.
3. Le repository demande au Data Source local d'enregistrer le nouveau statut pour l'identifiant de l'intervention.
4. L'état Riverpod est mis à jour immédiatement.
5. La liste et le détail affichent le nouveau statut sans modifier le JSON de référence.

3.3 Scénarios d'erreur

Situation	Traitement	Comportement UI
Réseau indisponible / timeout	Erreur de chargement typée	Message lisible + bouton « Réessayer ». Les statuts déjà enregistrés ne sont pas perdus.
JSON invalide	Erreur de désérialisation	Aucune donnée partielle affichée ; message d'erreur et possibilité de réessayer.
Erreur de stockage local	Erreur de persistance	Le changement n'est pas considéré comme sauvegardé ; message utilisateur.
Transition interdite	Règle métier	Aucune écriture ; l'UI ne propose normalement pas cette action.

3.4 Parcours utilisateur synthétique

Parcours principal

Ouverture → chargement → liste → recherche / filtres → détail → changement de statut → retour liste. L'onglet Profil reste accessible indépendamment du cycle des interventions.

États de chargement prévus

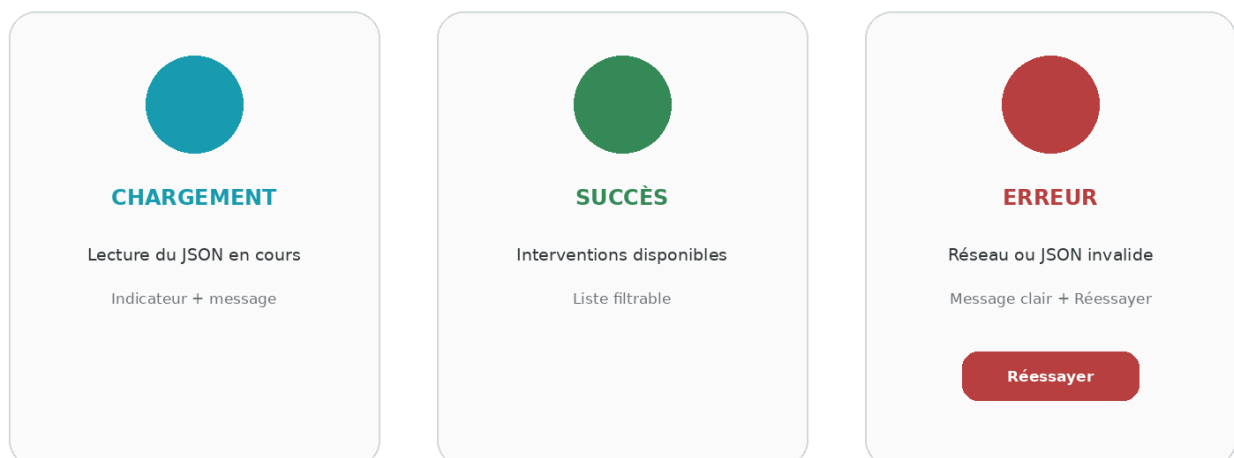


Figure 1 - États prévus lors du chargement des données.

4. Maquettes / wireframes des écrans clés

Les maquettes sont volontairement fonctionnelles : elles définissent la hiérarchie de l'information et le parcours, sans figer le design final. Le thème final utilisera Material 3 et le ColorScheme de l'application.

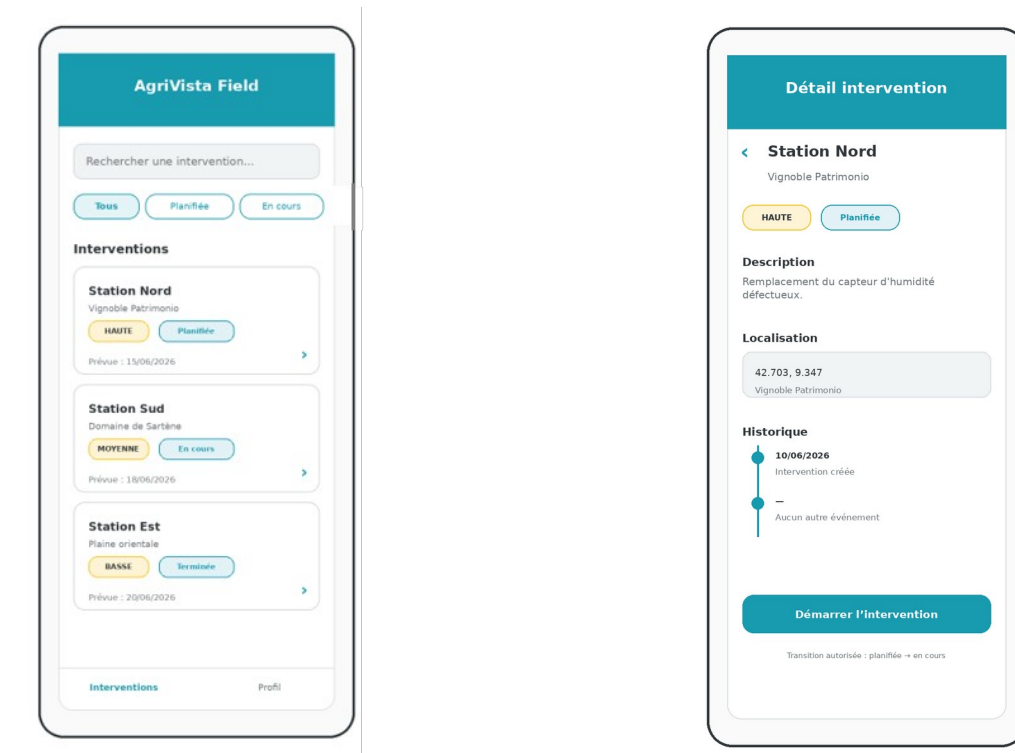
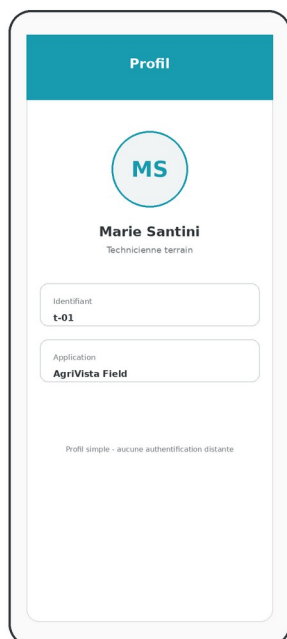


Figure 2 - Liste filtrable (gauche) et détail d'une intervention (droite).

Principes UX retenus

- La priorité et le statut sont visibles directement dans la liste.
- La recherche est accessible en haut de l'écran ; les filtres restent visibles.
- Le détail présente les informations avant l'action de changement de statut.
- Une intervention terminée n'affiche plus d'action de progression.
- Les états d'erreur proposent une action « Réessayer » explicite.



Écran Profil

Le profil reste simple, conformément au périmètre minimum. Il affiche l'identité du technicien provenant du JSON et le contexte de l'application. Aucune authentification distante n'est nécessaire dans cette version.

5. Modèle de données

Le modèle du Domain reprend les concepts du JSON sans dépendre de son format technique. Le statut local est modélisé séparément afin de respecter l'interdiction de modifier la source JSON.

Modèle de données du domaine

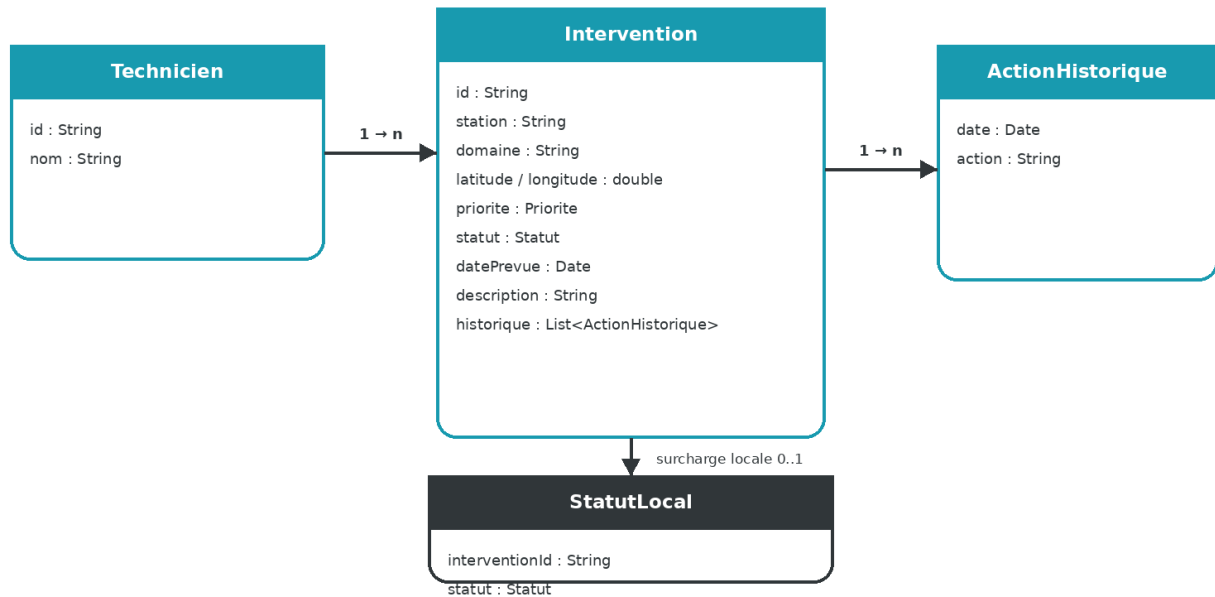


Figure 3 - Entités principales et surcharge locale du statut.

5.1 Entités et valeurs métier

Entité	Attributs principaux	Responsabilité
Technicien	id, nom	Identité du technicien fournie par le JSON.
Intervention	id, station, domaine, latitude, longitude, priorite, statut, datePrevue, description, historique	Objet métier principal.
ActionHistorique	date, action	Événement historique fourni par la source.
StatutLocal	interventionId, statut	Surcharge persistée localement après une modification.

5.2 Enumérations

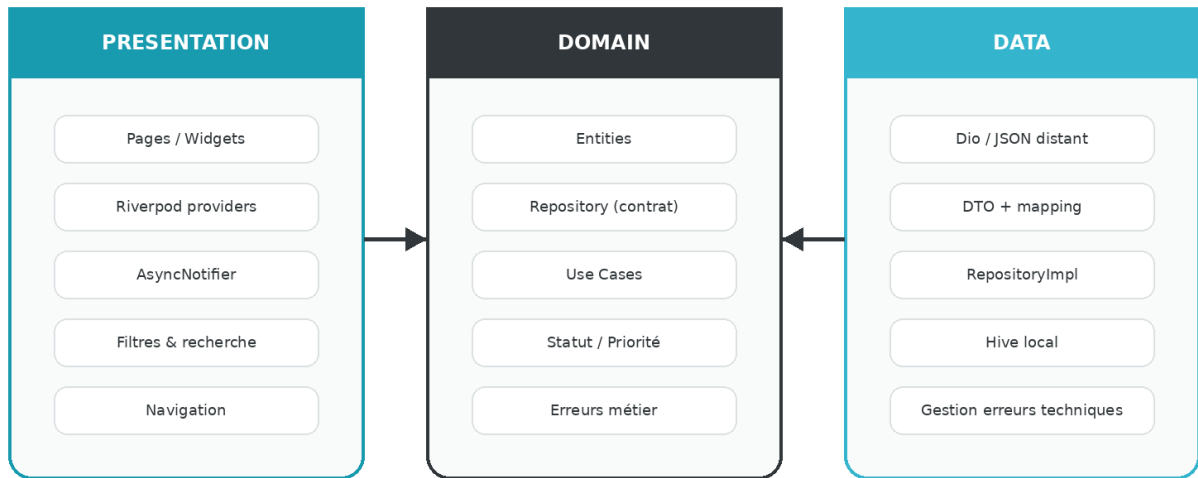
Type	Valeurs
Statut	planifiee · en_cours · terminee
Priorite	haute · moyenne · basse

5.3 Règles métier

- Une intervention possède exactement un statut et une priorité.
- Une transition de statut ne peut aller que vers l'état suivant.
- Le statut local, lorsqu'il existe, remplace le statut initial du JSON à l'affichage.
- L'historique fourni par le JSON reste en lecture seule dans le périmètre minimum.

6. Architecture logicielle et gestion d'état

Architecture cible - Clean Architecture



Règle de dépendance : Presentation et Data dépendent du Domain, jamais l'inverse.

Figure 4 - Découpage Data / Domain / Presentation.

6.1 Responsabilités des couches

Couche	Responsabilités	Dépendances
Presentation	Pages, widgets, navigation, AsyncNotifier, filtres/recherche, affichage loading/data/error.	Flutter + Riverpod
Domain	Entities, interfaces de repository, Use Cases, règles de transition et erreurs métier.	Dart pur
Data	Dio, lecture du JSON, DTO, mapping, Hive, implémentation du repository.	Détails techniques

6.2 Arborescence cible

```

lib/
├── core/
│   └── errors/
├── features/interventions/
│   ├── domain/
│   │   ├── entities/
│   │   ├── repositories/
│   │   └── usecases/
│   ├── data/
│   │   ├── datasources/
│   │   ├── models/
│   │   └── repositories/
│   └── presentation/
│       ├── providers/
│       ├── pages/
│       └── widgets/
├── features/profile/presentation/
└── main.dart
  
```

6.3 Choix Riverpod

Riverpod est retenu pour la gestion d'état et l'injection de dépendances. Un AsyncNotifier expose la liste des interventions et matérialise naturellement loading / data / error. Des providers dédiés portent les filtres et la recherche. Les repositories et Data Sources sont injectés via Provider afin de conserver un couplage faible et de faciliter les tests.

7. Choix techniques argumentés

Sujet	Choix	Justification	Alternative écartée
Client HTTP	Dio	Source JSON distante, timeouts, erreurs structurées et client injectable.	http : plus minimal mais moins pratique pour les besoins de robustesse.
Sérialisation	Freezed + json_serializable	Objets typés, immuables, fromJson/toJson générés.	Parsing manuel : plus de boilerplate et risque d'erreurs.
État / DI	Riverpod	Notion imposée/recommandée, AsyncValue, DI et testabilité.	BLoC : possible mais plus verbeux ici et nécessite une justification.
Persistence	Hive	Besoin simple clé → valeur par intervention, rapide et extensible.	SharedPreferences : possible mais moins adapté à des données métier ; Isar surdimensionné sans requêtes locales complexes.
UI	Material 3	Thème cohérent et conforme au périmètre.	Couleurs codées en dur : rejetées pour cohérence et maintenabilité.
Backend	Aucun	Le sujet impose le JSON de référence et exclut la synchronisation serveur.	Supabase/Firebase : hors périmètre de cette version.

7.1 Packages majeurs prévus

Package	Rôle
flutter_riverpod	Gestion d'état + injection de dépendances
dio	Lecture HTTP du JSON
freezed_annotation / freezed	Modèles immuables et unions
json_annotation / json_serializable	Désérialisation générée
build_runner	Génération de code
hive_flutter	Persistence locale des statuts

7.2 Erreurs typées

Les couches Domain/Data distingueront au minimum les échecs réseau, désérialisation et stockage local. L'objectif est d'éviter de transmettre des exceptions techniques brutes jusqu'aux widgets. La Presentation convertira ces échecs en messages compréhensibles et en actions adaptées.

Principe

Le chargement ne doit jamais rester bloqué : tout Future exposé à l'UI aboutit à un état data ou error. Le bouton Réessayer relance explicitement le chargement.

8. Stratégie de persistance locale des statuts

Flux de lecture et persistance des statuts

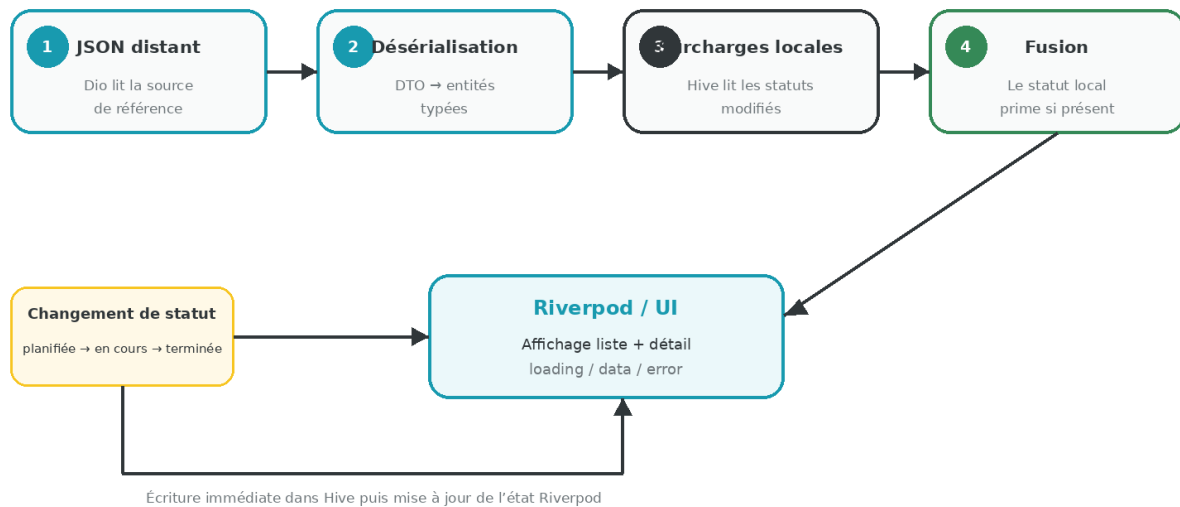


Figure 5 - Fusion de la source JSON avec les statuts locaux.

8.1 Structure locale

Une box Hive dédiée, par exemple « intervention_status », stockera une entrée par intervention modifiée. La clé est l'identifiant de l'intervention et la valeur est le statut courant. Exemple conceptuel : itv-1001 → en_cours.

8.2 Lecture au démarrage

1. Récupérer le JSON distant et créer les entités Intervention.
2. Lire la box Hive des statuts locaux.
3. Pour chaque intervention, rechercher une surcharge locale par son identifiant.
4. Si une surcharge existe, l'appliquer ; sinon conserver le statut issu du JSON.
5. Exposer la liste fusionnée via Riverpod.

8.3 Écriture lors d'une modification

1. Valider la transition dans le Domain.
2. Écrire immédiatement le nouveau statut dans Hive.
3. Mettre à jour l'état Riverpod seulement lorsque l'écriture a réussi.
4. Ne jamais modifier le fichier JSON distant.

8.4 Limites assumées de la première version

- La totalité du JSON n'est pas mise en cache local dans le périmètre minimum ; un premier chargement nécessite donc l'accès à la source.
- Les changements de statut locaux ne sont pas synchronisés vers un serveur central.
- L'historique fourni par le JSON n'est pas enrichi par les changements locaux dans le socle minimum.
- Une future version pourra ajouter un cache complet offline-first et une synchronisation bidirectionnelle.

9. Traçabilité des exigences et critères de validation

Exigence	Composants prévus	Critère de validation
Chargement JSON	Dio + RemoteDataSource + DTO	Chargement depuis l'URL sans modification de la source.
Liste + filtres + recherche	Presentation + providers de filtres	Résultats cohérents pour chaque statut/priorité et recherche.
Détail	InterventionDetailPage	Station, domaine, localisation, description, statut, historique visibles.
Progression du statut	Use Case UpdateStatus	Planifiée → en cours → terminée, pas de retour arrière.
Persistance locale	Hive + LocalDataSource	Le statut reste modifié après fermeture/redémarrage.
États chargement/succès/erreur	AsyncNotifier + AsyncValue.when	Chaque scénario est démontrable.
Thème & profil	Material 3 + ProfilePage	Palette cohérente et écran profil accessible.
Architecture	Data / Domain / Presentation	Aucun import Flutter/Dio/Hive dans le Domain.
Versionnement	Git	Commits réguliers, explicites et significatifs.

10. Conclusion de conception

La conception retenue privilégie le périmètre obligatoire et la maintenabilité. Le JSON distant demeure la source de référence en lecture seule ; les statuts modifiés sont superposés localement grâce à Hive. Le Domain reste indépendant des détails techniques, tandis que Riverpod assure à la fois l'injection de dépendances et l'état réactif de l'interface. Cette base permet de développer le socle demandé dans le temps imparti tout en conservant une évolution possible vers un cache complet et une synchronisation serveur.

Point de passage avant développement

Le développement devra respecter ce dossier : aucune fonctionnalité optionnelle ne sera prioritaire tant que la liste, le détail, le changement de statut, la persistance locale et les états de chargement ne sont pas validés.

Annexe - URL de la source de données

<https://utrera.ludovic.aflokkat-projet.fr/getInterventions.json>